Final Project Report

Project Abstract:

GrizzlyPaths is a web application that is meant to help upcoming IT majors by showing them a road map of skills that will be required for jobs in the desired industry. Our focus is on creating a road map that will display 10 job titles that will be displayed based on the occurrence within multiple job searching websites. As they choose the job, hard and soft skills will then be displayed based on importance from a variety of companies. Those skills will have corresponding courses at Georgia Gwinnett College that students must attend.

Team Members' Introductions and Group Photo

Team Members:

Sidibaba Simpara, William Chokbengboune, Charles Sarpong, Hieu Do

Client Presentation

Client: Krishan Bhalsod, Michelle Webb

Michelle: Previously worked on Grizzly Paths in Spring 2025. Attained an internship at UPS after presenting the project and is now working as a senior data analyst on the international marketing team at UPS.

Krishan: Designed the website and implemented Python logic for the Grizzly Paths project. Attained publications for my research project and my TAP project.

Roles

Name	Role
Sidibaba Simpara	UI/UX Leader, Team Manager
William Chokbengboune	Data Modeler, Client Liaison

Charles Sarpong	Testing Leader, Programmer
Hieu Do	Code Architecture, Documentation lead

Technologies Used

- **Frontend:** HTML/ CSS, Javascript
- **Backend:** Firebase
- **Project Management:** Jira
- **Version Control:** Git & Github
- **Project Dependencies:** React + Vite, Jest, Vitest, Boostrap, Paraphase, CSV-parser, Chart, Firebase SDK, Firebase CLI

Data Collections

- **Final Cleaned/Merged Dataset:** https://raw.githubusercontent.com/GGC-SD/GrizzlyPaths/refs/heads/main/docs-Spring2025/final_files/merged_jobs_cleaned%20(6).csv
- **Course.csv:** https://github.com/GGC-SD/GrizzlyPaths/blob/main/docs-Fall2025/grizzlypaths/src/Component/Course.csv

---

Installation:

1. Install Visual Studio Code
2. Install Git
3. Install Node.js
    - **3.1: Verify installation**
        - node -v
        - npm -v
4. **Clone the repository:**
    - git clone https://github.com/GGC-SD/GrizzlyPaths.git
    - cd GrizzlyPaths/docs-Fall2025
5. **Install project dependencies (via VSCode):**
    - React + Vite: npm create vite@latest my-react-app -- --template (Replace my-react-app with your desired project name)

- React-router-dom: npm install react-router-dom
- Bootstrap: npm install bootstrap@5.3.3
- Bootstrap-icon: npm install bootstrap-icons
- React-chartjs-2: npm install react-chartjs-2
- Paraphase: npm install papaparse
- Csv-parser: npm install csv-parser
- Firebase CLI globally for deployment: npm install -g firebase-tools
- Install Firebase SDK in the project: npm install firebase
- Vitest: npm install vitest --save-dev
- Jest: npm install jest --save-dev
- Chart-js: npm install --save chart.js react-chartjs-2

6. **Run the project:**
   - npm run dev
7. **Run the test:**
   - npm run test

Usage

1. Log in to the website. If you don't have an account, click the link to create one. If you have an account but you forgot your password, click the link to reset password.
2. In the dashboard, you can change the major by using the dropdown in the top left part of the page.
3. You can click on the Recommend Course link to see 3000/4000 level courses for the each major.
4. You can click on the Roadmap to see the visualize of the job posting for each major. When you click the major that you want to choose, it will show the job posting. When you click the job posting, it will show the technical skill of that job postings. Then, you can click on the technical skill to show the classes that you need to take. To change jobs, click the arrow in the bottom of the screen to reload the wheel. The wheel will then disappear and only show the majors.
5. The logout is located in the upper right corner of the Dashboard.

Code Documentation

Login.jsx

```
/**
 * Login Component
 * ----------------
 * This component handles user authentication using Firebase Authentication
 * and retrieves associated student data stored in Firebase Realtime Database.
 *
 * Workflow:
 * 1. User enters email + password.
 * 2. signInWithEmailAndPassword() authenticates the user.
 * 3. Firebase returns a userCredential containing the UID.
 * 4. Using the UID, we retrieve the student's profile info from:
 *        Realtime Database → "student/<uid>"
 * 5. If data exists:
 *        - Save student info to localStorage
 *        - Call onLogin() to update the app state
 *    If not:
 *        - Display "Student data not found"
 *
 * Props:
 * - onLogin (function): Callback executed after successful login and data retrieval.
 *
 * State Variables:
 * - email: Stores the user's email input
 * - password: Stores the user's password input
 * - message: Stores success/error messages for UI feedback
 */

import { useState } from "react";
import { signInWithEmailAndPassword } from "firebase/auth";
import { getDatabase, ref, get } from "firebase/database";
import { auth } from "../Backend/firebase";

export default function Login({ onLogin }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("");

  /**
   * Handles form submission:
   * - prevents page reload
   * - performs Firebase Auth login
   * - retrieves student data from Realtime Database
   * - stores user info locally
   */
  const handleSubmit = async (e) => {
    e.preventDefault();
    setMessage("");
    try {
      // Authenticate the user with Firebase Auth
      const userCredential = await signInWithEmailAndPassword(auth, email, password);
      const uid = userCredential.user.uid;
      await auth.currentUser.getIdToken(true);

      // Retrieve student info from Realtime Database
```

Signup.jsx

```
/**
 * SignUp Component
 * ----------------
 * This component handles user registration using Firebase Authentication
 * and stores additional profile information inside Firebase Realtime Database.
 *
 * Workflow:
 * 1. User enters required fields:
 *      - Full Name
 *      - Email
 *      - Password
 *      - Student ID
 *      - Major
 *
 * 2. createUserWithEmailAndPassword() creates a new Firebase Auth user.
 * 3. On success, Firebase returns a userCredential containing the UID.
 * 4. Using the UID, the component stores additional user profile data at:
 *        Realtime Database → "student/<uid>"
 *
 * 5. Displays a confirmation message and redirects the user to login.
 *
 * State Variables:
 * - name: User's full name
 * - email: User's email address
 * - password: Account password
 * - studentID: Numeric or alphanumeric student ID
 * - major: User's selected major from dropdown list
 * - message: Status or error message for UI feedback
 *
 * Navigation:
 * - Uses react-router-dom's useNavigate() to redirect after successful signup.
 *
 * Firebase Services Used:
 * - Firebase Authentication → createUserWithEmailAndPassword()
 * - Firebase Realtime Database → set() to store student data
 */
```

For more details: https://github.com/GGC-SD/GrizzlyPaths/blob/main/docs-Fall2025/Documents/Developer/Coding%20Documentation.docx

---

Testing:

- For testing, we used vitest and jest. The vitest was used to mock the environment while the jest used to test the data inputs. Vitest was going to be the only one used but it was deprecated for the version of vite we used which lead to the installation of jest. Usually, most of the team members run the test and it passed. However, one team member did not pass 1 test in 14 tests. The reason can be different operating systems. In the future, we will try to resolve the issue as soon as possible and avoid such an incident from happening again.

```
vi.mock('../src/Backend/firebase', () => ({
  db: {}, // We just need a dummy object here
}));

// 2. Mock the Firebase Database SDK methods
vi.mock('firebase/database', () => {
  return {
    ref: vi.fn(),
    push: vi.fn(() => ({ key: 'mockedCourseId' })), // Mock push returning a key
    set: vi.fn(() => Promise.resolve()), // Mock set returning a resolved promise
  };
});

describe('AddCourseForm', () => {

  // Clear mocks before each test to ensure clean state
  beforeEach(() => {
    vi.clearAllMocks();
  });

  test('renders input fields and button', () => {
    render(<AddCourseForm />);
    expect(screen.getByLabelText(/course name/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/course number/i)).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /add course/i })).toBeInTheDocument();
  });

  test('shows validation message if fields are empty', () => {
    render(<AddCourseForm />);

    // Note: The HTML 'required' attribute might stop submission in a real browser,
    // but in JSDOM/React Testing Library, fireEvent can bypass it or trigger the React handler.
    fireEvent.click(screen.getByRole('button', { name: /add course/i }));

    // Your component manually checks: if (!courseName.trim() || !courseNumber)
    expect(screen.getByText(/please enter both a course name and number/i)).toBeInTheDocument();
  });

  test('submits valid course and shows success message', async () => {
    // Import the mocked 'set' to check if it was called
    const { set } = await import('firebase/database');
```

Locate Installation, Developer, User Documentation

Installation Documentation:

https://github.com/GGC-SD/GrizzlyPaths/tree/main/docs-Fall2025/Documents/Installation

Developer Documentation:

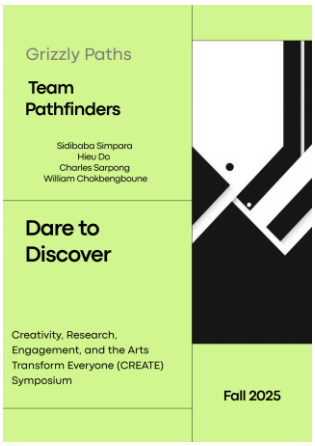
https://github.com/GGC-SD/GrizzlyPaths/tree/main/docs-Fall2025/Documents/Developer

User Documentation:

https://github.com/GGC-SD/GrizzlyPaths/tree/main/docs-Fall2025/Documents/User

---

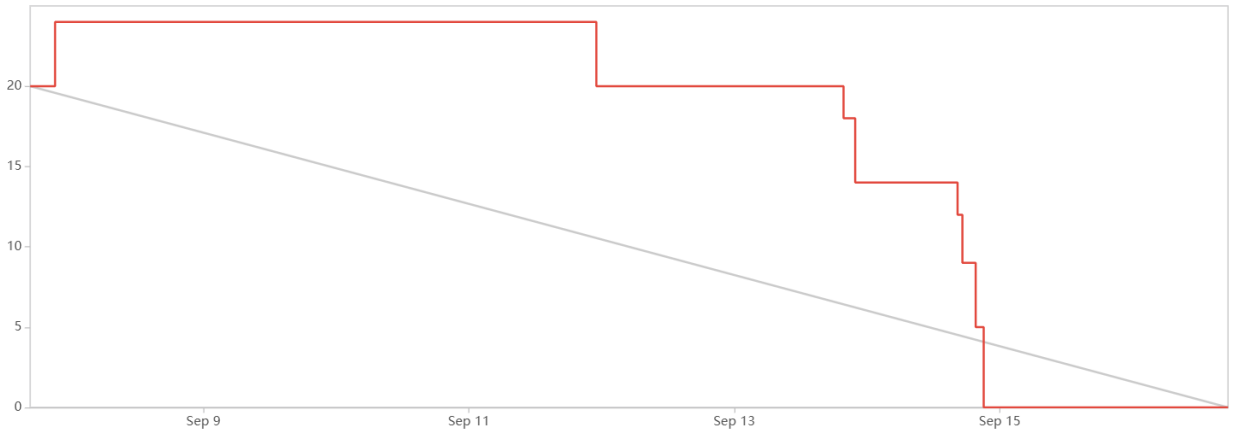
Team Flyer

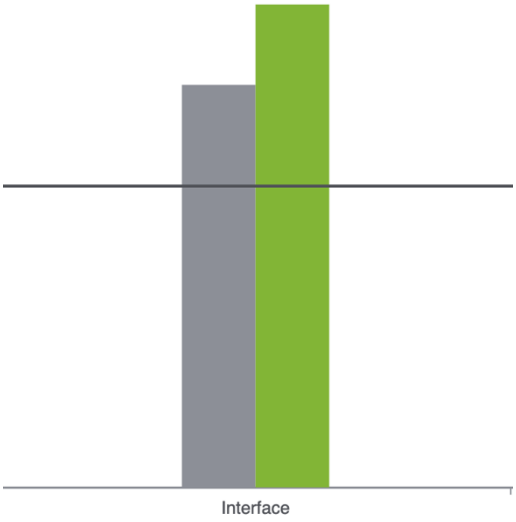


Live Demo Link: https://www.youtube.com/watch?v=qzcVMQp8404

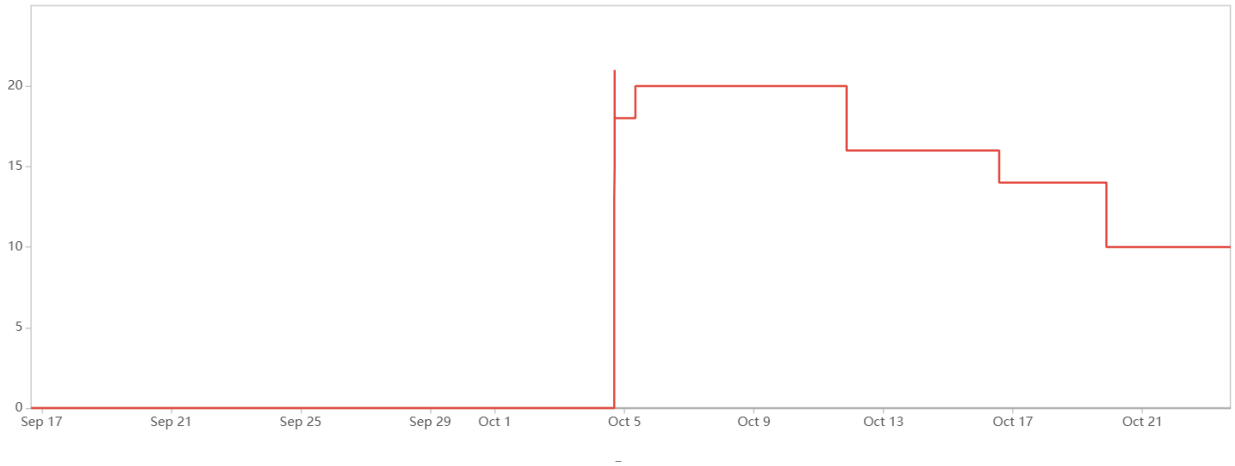
Jira DashBoard + Burndown/Velocity Charts

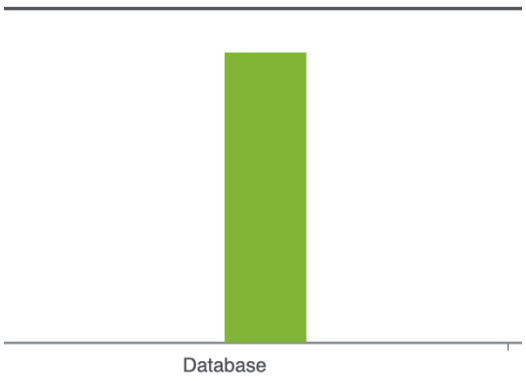
Sprint 1:



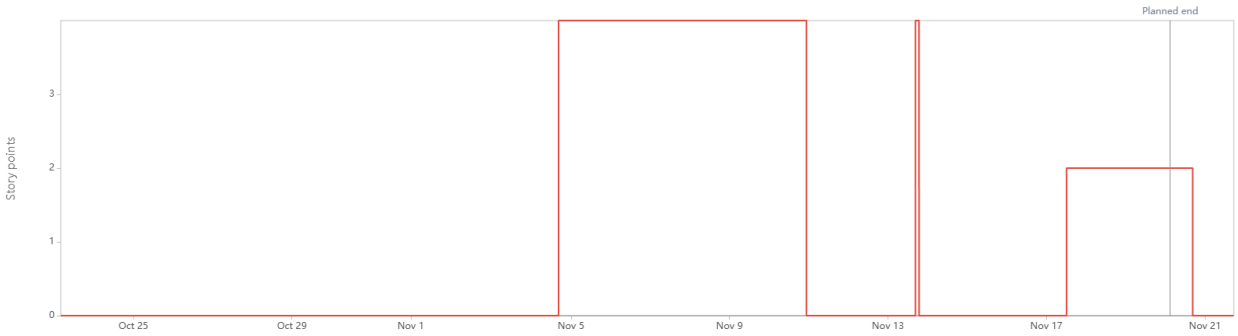


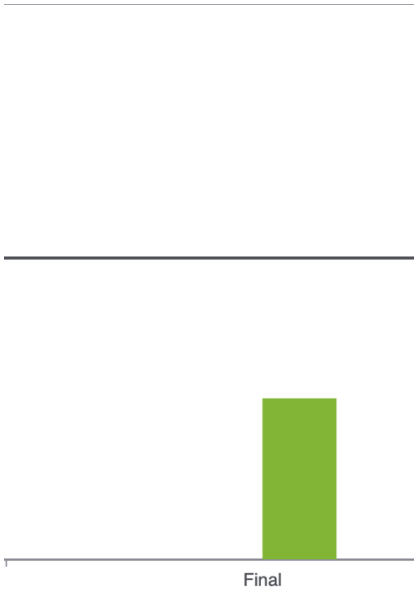
Sprint 2:




Database

Sprint 3:


Planned end
Story points
3
2
1
0
Oct 25
Oct 29
Nov 1
Nov 5
Nov 9
Nov 13
Nov 17
Nov 21



Summary:

Throughout the project lifespan, we faced many challenges that we have never really knew how to confront. We learned many things we never expected, and we experienced many frustrations that we have forgotten or wanted to forget.

Jira was a tough challenge to learn. Even now, we still do not have a complete grasp on it. We don't know how to measure the complexity of each user story to accurately determine the difficulty of each task. Some of us still don't know how to properly formulate a user story or even the questions to ask them.

The process was very informative though. Throughout the project, we learned how to apply familiar concepts to foreign computing languages, use our resources, break up tasks, organize file structures, formulate plans and narrow down technologies more efficiently.

If we had to redo it, we would take more initiatives in the project. Starting earlier would help a lot. During the first iteration, we seemingly procrastinated it which led to poor execution of the project. After that, we didn't try to ask or understand the project.

Summary of Each Iteration's Results

Iteration 1:

For the first iteration, we have difficulty to understand the website on React. We have done to show the student's basic information when students logged in by their studentID and

name. At the end of iteration 1, html pages are set, and layout is made. The dashboard was showing all information, which is not good for the website.

Iteration 2:

After understanding React and how it works, we need to transform the code that we have into the React. Sidibaba continue to work on UI. We also start working on the database. Hieu was responsible to handle the log in page. William and Charles will focus on the roadmap. At end of iteration 2, database is established. Roadmap is remade according to client's wish.

Iteration 3:

At the end of iteration 3, we standardized the format of the page to make everything as consistent as we could. The log in page now had full function. The roadmap was redesigned to be easier to read and more friendly. The admin board was reimplemented, but it is not finish yet.

Features Implemented

- Firebase - Log in page with full function.
- Show the general of student information in dashboard.
- There is a general list of 3000/4000 recommended courses in each major.
- Have a chart to show to top 5 jobs for each major.
- Import button for cleaned csv of merged data is in place.

Known Issues

- Loading of data to "roadmap" is inconsistent and can take too long.
- At random times, page refreshes on load.
- Data needs to be cleaned further.

TODO Tasks

- Admin authentication must be implemented in further iterations of project.
- Visualization of information can be improved if necessary.
- Page formatting needs to be improved.

- Webscraping and updating button must be added to admin box.
- Course history section that has interact ability with roadmap needs to be added to student.
- Main page needs to be separated into 3 view for student, admin, and guest, so that login isn't necessary.

Group Survey:

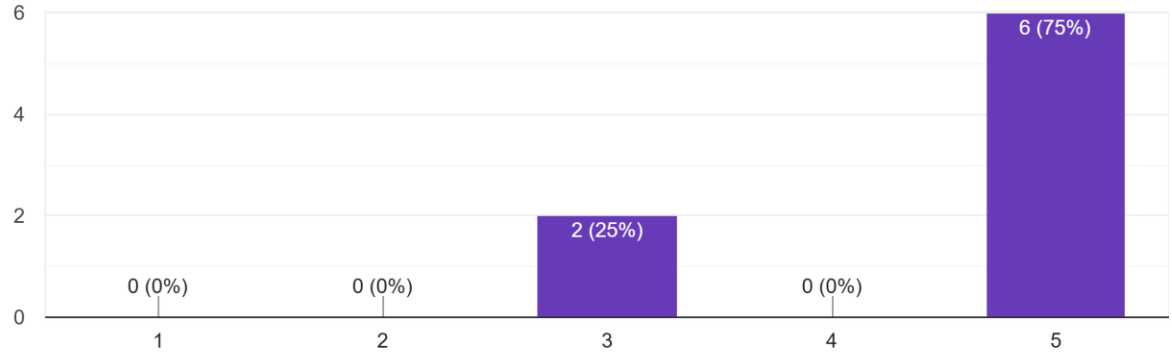
1. What did you like the most about this project?
   The sunburst chart from the last group provided a good visualization of the information.
2. How do you feel about the current state of this project?
   It's good but unfinished. The UI could be improved.
3. What needs the most improvement?
   The UI lacks features, so it is very barebone. The technical skills need more research. Contact with outside sources would prove helpful. Automatic updating of information would be nice.
4. What new feature or capability would you be most excited to see added to GrizzlyPaths?

- **Content and Accuracy:** Users would be excited to see more fields and more accurate course features with relevant skills, and the addition of course overviews or descriptions.
- **Functionality:** Automated web scraping and the display of the percentage of courses are desired new capabilities.
- **Personalization:** The ability for customization is a requested feature.
- **Other feedback:** One response indicated that the product is "perfect," while another was "not sure" about new features.

5. On a scale of 1-5, how would you rate the user interface and ease of navigation for GrizzlyPaths?

On a scale of 1-5, how would you rate the user interface and ease of navigation for GrizzlyPaths?

8 responses

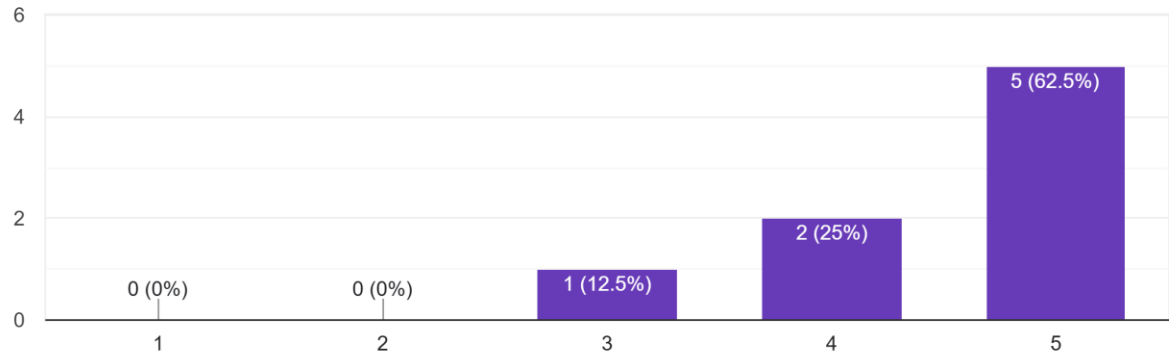


Average rating is a 4.5

6. How would you rate the effectiveness of GrizzlyPaths in meeting its primary goal?

How would you rate the effectiveness of GrizzlyPaths in meeting its primary goal?

8 responses



Intellectual Property

https://github.com/GGC-SD/GrizzlyPaths/blob/main/docs-Fall2025/Documents/Developer/Intellectual%20Property%20Contract%20update%20with%20merge%20(1).pdf

---